

Guide to GitHub Copilot

Amund Alsvik

Jørgen Vestly

July 2026

1 Introduction

This document serves as a guide for using GitHub Copilot. It introduces some tips and tricks for making it an efficient tool for scientific projects through prompts and coding examples. Then, it further addresses some caveats and limitations for using GitHub Copilot.

2 Using GitHub Copilot

GitHub Copilot is an AI-assisted programming tool that can help with code completion, explanation, debugging, testing, refactoring, documentation, and larger project edits. It is integrated into development environments such as Visual Studio Code, where it can appear as inline suggestions while writing code, as Copilot Chat for questions and explanations, as inline chat for local edits, and as Copilot Edits for larger changes across files.

This chapter is written as a practical guide to using the tool. The goal is not to treat Copilot as an oracle or as an autonomous scientist. Rather, the goal is to show how it can be used as a disciplined assistant in programming and scientific work. Copilot can help turn an idea into code, code into tests, tests into plots, and plots into documentation. The user is still responsible for the scientific formulation, the assumptions, the verification, and the final interpretation.

2.1 Getting access and signing in

To use GitHub Copilot, the student needs a GitHub account with access to a Copilot plan. This may be through an individual Copilot plan, Copilot Free with limited usage, an organization or university subscription, or student access through GitHub Education [3, 1, 2]. The exact access route may differ between students, so the first step is simply to check that Copilot is enabled for the GitHub account that will be used in the editor.

In Visual Studio Code, Copilot is used through the GitHub Copilot extension and the built-in Copilot Chat interface. After installing VS Code, open the Extensions view and search for *GitHub Copilot*. Install the official extension from GitHub, then sign in with the GitHub account that has Copilot access. VS Code allows sign-in from the Accounts menu, from the Copilot menu in the status bar, or by running the command `GitHub Copilot: Sign in` from the command palette [5].

A quick way to check that Copilot works is to open a Python file and write a precise comment:

```
# Compute the fidelity  $|\langle \psi_{\text{target}} | \psi \rangle|^2$ 
# between two normalized complex state vectors.
```

If Copilot is active, it should suggest a completion in grey text. The suggestion can usually be accepted with **Tab**. The exact keybindings can depend on the editor and operating system, but the important point is to begin with a small, local test before relying on Copilot in a larger project.

When Copilot does not appear to work, the first things to check are whether the correct GitHub account is signed in, whether that account has Copilot access, whether the extension is enabled, and whether Copilot is disabled for the current workspace. If several GitHub accounts are used, for example one personal account and one university account, make sure that VS Code is using the account with Copilot access.

2.2 A useful mental model

A useful way to think about Copilot is that it responds to context. It sees the file being edited, the selected code, nearby comments, function names, open files, and information explicitly added to the prompt. In chat and edit workflows, it may also use files that are attached to the working set. This means that Copilot is strongly affected by how the user writes the surrounding code and how clearly the task is described.

For example, a function called

```
def process(data):
```

does not give Copilot much information. A function called

```
def estimate_rabi_frequency_from_excited_state_population(time, p_excited):
```

already tells Copilot the scientific setting, the likely input variables, and the kind of computation that may be needed. Clear names, small functions, docstrings, and local comments are therefore not only good for human readers. They also improve the context that Copilot can use.

The safest way to use Copilot is to ask for small, inspectable steps. A large prompt such as “write my whole project” often produces code that looks coherent but is difficult to verify. A better workflow is to ask for a plan, then implement one function, then generate tests, then integrate the function into the larger program.

2.3 Inline suggestions

Inline suggestions are the completions that appear while typing. They are useful for local tasks: completing a loop, writing a small helper function, filling in repetitive code, adding a docstring, or completing a plotting routine. Inline suggestions are fast, but they are not a substitute for planning.

A good trick is to write a short comment before the code. For example:

```
# Compute the state fidelity |⟨psi_target|psi⟩|^2  
# for two normalized complex state vectors.
```

This gives Copilot a precise local objective. A likely completion is a small function that computes an inner product and returns the squared absolute value. The user should still inspect whether the convention is correct, for example whether the first argument is conjugated correctly.

Inline suggestions are best when the user knows what the next few lines should do. When the task requires explanation, comparison, or debugging, Copilot Chat is usually better.

2.4 Copilot Chat

Copilot Chat is useful when the user wants to ask a question rather than complete the next line. It can explain unfamiliar code, suggest a design, identify possible bugs, write tests, or compare two im-

plementations. A good chat prompt contains the task, the context, the constraints, and the desired output.

A weak prompt is:

Write quantum code.

A stronger prompt is:

Write a small Python function that simulates a driven two-level system with Hamiltonian

$$H(t) = \frac{\Delta}{2}\sigma_z + \frac{\Omega(t)}{2}\sigma_x.$$

Use NumPy and SciPy only. The function should return the final state after a piecewise-constant pulse. Do not use Qiskit or QuTiP. Include a short docstring explaining the units assumed.

The stronger prompt tells Copilot the physical model, the language, the libraries, the output, and the restrictions. This makes the answer easier to inspect.

A useful general prompt pattern is:

I am trying to [task]. The scientific context is [context]. Use [language and libraries]. Produce [specific output]. Do not [constraints]. Before writing code, state the assumptions.

The final sentence is important in scientific programming. Many errors come from hidden assumptions about units, normalization, boundary conditions, array shapes, or noise models.

2.5 Inline chat

Inline chat is useful when the user has selected a specific block of code and wants a local change. For example, one can highlight a function and ask:

Add type hints and a NumPy-style docstring to the selected function. Do not change the implementation.

or

Explain the selected function line by line. Focus on the mathematical meaning, not only the Python syntax.

or

Review the selected code for possible bugs involving complex conjugation, normalization, and array shapes. Do not rewrite the code yet.

Inline chat is safer than asking for a large multi-file change, because the scope is explicit. The user can see exactly what code is being discussed.

2.6 Copilot Edits, edit mode, and agent mode

For larger tasks, Copilot can be used through edit workflows. In edit mode, the user gives Copilot a task and a working set of files. Copilot then proposes changes across those files. This is useful when the

change is larger than one function but still controlled, such as adding a command-line argument, moving plotting code into a separate module, or updating a README after changing a script.

Agent mode is more autonomous. It is designed for tasks where Copilot may decide which files to edit and may suggest terminal commands, for example running tests or checking whether an error has been fixed. This can be useful, but it also requires more discipline from the user. The user should inspect the proposed diff, check which files were changed, run the tests, and reject changes that modify unrelated behavior.

A good edit-mode prompt is narrow:

Add a command-line argument `-output-dir` to the analysis script. Use it only for saving figures and parameter files. Keep all existing defaults unchanged. Update the README with one example command.

A risky prompt is broad:

Clean up the whole project.

The second prompt gives Copilot too much freedom. In scientific code, broad edits may silently change numerical behavior. A better instruction is:

Refactor the script into smaller functions. Keep the numerical behavior unchanged. Do not change the equations, constants, or default parameters. After editing, summarize exactly which files changed.

2.7 Custom instructions

For a course repository or research project, it is useful to give Copilot persistent instructions. A common project-level file is:

```
.github/copilot-instructions.md
```

This file should contain short, concrete statements about coding style, libraries, testing, documentation, and scientific conventions. For a quantum-science repository, one might write:

```
# Copilot instructions for this repository

Use Python 3.11 or newer.
Prefer NumPy, SciPy, matplotlib, and pytest.
Use pathlib for file paths.
Use type hints for public functions.
Write NumPy-style docstrings.
Do not introduce new dependencies unless explicitly requested.
For quantum states, use complex NumPy arrays.
State whether state vectors are assumed to be normalized.
For Hamiltonians, state the units used for time, frequency, and hbar.
For numerical routines, include simple sanity checks.
For plotting, label axes and include units when known.
For tests, include at least one case with an analytically known answer.
```

The point is not to make the instruction file long. The point is to make recurring expectations explicit. If the project always uses NumPy and SciPy rather than QuTiP, Copilot should be told. If all quantum states are represented as column vectors or one-dimensional arrays, Copilot should be told. If plots must include units, Copilot should be told.

2.8 Prompt files

Prompt files are reusable prompts stored in the workspace. They are useful when the same task is repeated many times, such as reviewing code, generating tests, writing documentation, or checking scientific assumptions. A prompt file is usually placed in a directory such as:

```
.github/prompts/
```

For example, one could create:

```
.github/prompts/review-quantum-code.prompt.md
```

with content such as:

```
Review the selected quantum-simulation code.
```

```
Focus on:
```

- normalization of state vectors
- Hermiticity of Hamiltonians
- correct use of complex conjugation
- units of time and frequency
- array shapes
- numerical stability
- missing tests

```
Do not rewrite the code yet.
```

```
First list possible issues.
```

```
Then suggest minimal changes.
```

The user can then run this prompt when reviewing a new function. This makes Copilot behave more consistently. Instead of improvising a new review prompt each time, the project has a reusable review procedure.

A second useful prompt file could be:

```
.github/prompts/write-quantum-tests.prompt.md
```

with content such as:

```
Write pytest tests for the selected function.
```

```
Use simple quantum systems where the expected result is known analytically.
```

```
Include tests for:
```

- normalization
- shape of returned arrays
- a zero-pulse or identity-evolution case
- invalid input dimensions

```
Do not change the function implementation.
```

```
Explain what each test checks.
```

This follows a good scientific workflow: implementation should be accompanied by tests that encode known behavior.

2.9 Prompting strategies

Good prompting is not about writing long prompts. It is about reducing ambiguity. The most useful prompts often ask Copilot to separate planning, implementation, testing, and interpretation.

A planning prompt is:

Before writing code, outline the computational workflow. Identify the input data, the main functions, the assumptions, and the tests I should run.

A minimal-change debugging prompt is:

This code runs, but the final state norm is not one. Find the most likely cause. Suggest the smallest possible fix. Do not refactor unrelated code.

A scientific-review prompt is:

Review this implementation as scientific code. Focus on whether the equations, units, array shapes, and numerical assumptions are consistent. Do not rewrite the code yet.

A test-generation prompt is:

Write pytest tests for this function. Include one case where the analytical answer is known. Include one invalid-input case. Explain why each test is useful.

A documentation prompt is:

Write a docstring for this function. Explain the physical model, the meaning of each parameter, the units, and the returned quantity.

A comparison prompt is:

Give two implementations. The first should be simple and readable. The second should be more efficient for repeated simulations. Explain the tradeoff.

These prompts encourage Copilot to support reasoning rather than only generate code.

2.10 Larger example: Copilot for quantum-system discovery

We now consider a larger example where Copilot is used in a scientific-discovery workflow for a quantum system. The goal is not to let Copilot independently discover physics. The goal is to use Copilot to help organize a reproducible workflow for testing hypotheses about an unknown or partially known quantum system.

Suppose we have experimental data from a driven two-level system. The data consists of pulse duration t and measured excited-state probability $P_1(t)$. A simple model is Rabi oscillation under a driven qubit Hamiltonian,

$$H = \frac{\Delta}{2}\sigma_z + \frac{\Omega}{2}\sigma_x,$$

where Δ is a detuning and Ω is a drive amplitude. On resonance and without decoherence, one expects oscillatory population transfer. With detuning, imperfect contrast, or decoherence, the measured curve may deviate from the ideal model. The scientific question is whether the data are consistent with a simple coherent Rabi model, or whether a damped or biased model is needed.

A bad prompt is:

Discover the Hamiltonian from this data.

This prompt is too vague. It does not specify the candidate models, the measured observable, the noise assumptions, or the validation criterion. A better approach is to use Copilot step by step.

2.10.1 Step 1: ask for a workflow

The first prompt should request a plan rather than code.

I have experimental data from a driven two-level quantum system. The CSV file has columns `time` and `p1`, where `p1` is the measured excited-state probability. I want to test whether the data are consistent with a simple Rabi model, or whether damping and offset terms are needed. Propose a scientific Python workflow. Include data loading, candidate models, fitting, validation, plotting, and tests. Do not write code yet.

The answer should be inspected before implementation. A reasonable workflow might include loading the data, plotting the raw signal, fitting an ideal Rabi model, fitting a damped Rabi model, comparing residuals, and validating on held-out pulse durations. The student should check whether the proposed workflow matches the actual experiment.

2.11 Step 2: ask Copilot to define candidate models

After the workflow is clear, ask for small model functions. A useful prompt is:

Write two Python functions for candidate models of excited-state probability. The first should be an ideal Rabi model

$$P_1(t) = A \sin^2(\Omega t/2) + B.$$

The second should be a damped Rabi model

$$P_1(t) = B + Ae^{-t/T} \sin^2(\Omega t/2).$$

Use NumPy. Include type hints and docstrings. Do not fit the models yet.

Copilot may produce:

```
import numpy as np

def ideal_rabi(t: np.ndarray, amplitude: float, omega: float, offset: float) -> np.ndarray:
    """
    Ideal Rabi-oscillation model for the excited-state probability.

    Parameters
    -----
    t:
        Pulse duration.
    amplitude:
        Oscillation contrast.
    omega:

```

```

        Rabi angular frequency in inverse time units.
    offset:
        Constant probability offset.

    Returns
    -----
    np.ndarray
        Model prediction for the excited-state probability.
    """
    return offset + amplitude * np.sin(0.5 * omega * t) ** 2

def damped_rabi(
    t: np.ndarray,
    amplitude: float,
    omega: float,
    decay_time: float,
    offset: float,
) -> np.ndarray:
    """
    Damped Rabi-oscillation model for the excited-state probability.

    The damping factor is  $\exp(-t / \text{decay\_time})$ .
    """
    return offset + amplitude * np.exp(-t / decay_time) * np.sin(0.5 * omega * t) ** 2

```

The code is simple, but the scientific choices matter. The model assumes a particular functional form, treats damping phenomenologically, and ignores possible state-preparation and measurement errors except through the offset and amplitude. These assumptions should be stated in the report.

2.12 Step 3: ask for fitting code

The next prompt should be constrained. For example:

Write a function that fits a model function to `time` and `p1` data using `scipy.optimize.curve_fit`. The function should accept the model, initial parameter guesses, and parameter bounds. Return the fitted parameters, covariance matrix, predictions, residuals, and mean squared error. Include input validation.

A possible implementation is:

```

from collections.abc import Callable
from scipy.optimize import curve_fit

def fit_probability_model(
    model: Callable,
    time: np.ndarray,
    p1: np.ndarray,
    p0: list[float],
    bounds: tuple[list[float], list[float]],
) -> dict:

```

```

"""
Fit a probability model to excited-state population data.
"""
if time.ndim != 1 or p1.ndim != 1:
    raise ValueError("time and p1 must be one-dimensional arrays.")

if len(time) != len(p1):
    raise ValueError("time and p1 must have the same length.")

if np.any((p1 < 0.0) | (p1 > 1.0)):
    raise ValueError("Measured probabilities should lie between 0 and 1.")

params, covariance = curve_fit(
    model,
    time,
    p1,
    p0=p0,
    bounds=bounds,
)

prediction = model(time, *params)
residuals = p1 - prediction
mse = float(np.mean(residuals**2))

return {
    "params": params,
    "covariance": covariance,
    "prediction": prediction,
    "residuals": residuals,
    "mse": mse,
}

```

This is a typical place where Copilot is useful. It knows the mechanics of using `curve_fit`, but the user must choose meaningful initial guesses, bounds, and model assumptions. For example, Ω must be in units consistent with the time data, and T should be positive.

2.13 Step 4: ask Copilot for a scientific review

Before trusting the fit, ask Copilot to review the assumptions.

Review this fitting workflow as scientific code. Focus on parameter bounds, units, identifiability, overfitting, and whether the damped model is being compared fairly to the ideal model. Do not rewrite the code yet.

This type of prompt is valuable because it asks Copilot to act as a reviewer. It may point out that the damped model has more parameters, so a lower training error is not automatically evidence of a better physical model. It may also suggest held-out validation, residual plots, or information criteria.

2.14 Step 5: ask for validation

A useful validation prompt is:

Modify the analysis workflow so that every fourth data point is held out for validation. Fit the models on the remaining data and report both training and validation mean squared error. Keep the model functions unchanged.

The phrase *keep the model functions unchanged* protects the scientific object being tested. Copilot should change the workflow, not silently change the equations.

2.15 Step 6: ask for plots

Scientific discovery requires visual inspection. A good plotting prompt is:

Write a plotting function that shows the measured excited-state probabilities, the ideal Rabi fit, and the damped Rabi fit. Use different markers for training and validation points. Add a second panel showing residuals for both models. Label the axes and save the figure to a file. Do not call `plt.show()`.

The plot is not just decoration. It reveals whether the model fails systematically, for example at long pulse times, near maxima, or near minima.

2.16 Step 7: ask for tests

Before using the code in a report, ask for tests.

Write pytest tests for the model functions and fitting utility. Include a test where synthetic data are generated from the ideal Rabi model with known parameters and no noise. Check that the fitted parameters recover the known values approximately. Also test that invalid probabilities raise a `ValueError`.

This is one of the most productive uses of Copilot. It turns the scientific assumptions into executable checks.

2.17 Step 8: ask for a careful summary

After running the analysis, Copilot can help draft a summary, but only from actual results.

Using the fitted parameters, training error, validation error, and residual plots, draft a short scientific summary. State whether the ideal or damped Rabi model is better supported. Do not claim that the Hamiltonian has been uniquely identified. Mention limitations and possible next experiments.

The instruction *do not claim that the Hamiltonian has been uniquely identified* is important. Data from one observable and one experiment may support a model without proving uniqueness. Copilot can help write the paragraph, but the user must decide what the evidence actually supports.

2.18 What the quantum example teaches

This example illustrates how Copilot can support scientific discovery without replacing scientific judgment. The human defines the physical question, chooses candidate models, checks units, decides how to validate, and interprets the result. Copilot helps implement the workflow, generate tests, create plots, review assumptions, and document the analysis.

The best use of Copilot is not to ask for a final answer immediately. It is to make the workflow more explicit. A good session moves through planning, model definition, implementation, validation, plotting, testing, and interpretation. At each stage, Copilot can assist, but each output must be inspected.

In quantum applications, this discipline is especially important. A simulation can run while using the wrong convention for $\hbar$, the wrong sign in the Hamiltonian, an unnormalized state vector, or an inconsistent unit for time. Copilot may not catch these mistakes unless the prompt asks it to look for them. Good prompts therefore make the scientific assumptions visible.

2.19 Declaring and documenting Copilot use

Students should be transparent about how they use Copilot in a course project, report, or thesis-related workflow. The purpose of such a declaration is not to punish the use of the tool. The purpose is to make the workflow reproducible and to distinguish between human scientific judgment and AI-assisted implementation. This is especially important in scientific programming, where generated code may run correctly as Python while still implementing the wrong model, using inconsistent units, or hiding an invalid assumption.

A useful declaration should say what Copilot was used for, what it was not used for, and how the output was verified. For example, it is reasonable to write that Copilot was used for boilerplate code, plotting routines, debugging suggestions, docstrings, or unit tests. It is also useful to state that all generated code was manually inspected, tested on simple cases, and modified by the student where necessary. The declaration should not imply that Copilot is a co-author or that it is responsible for the scientific conclusions.

A short declaration could be:

GitHub Copilot was used as a programming assistant during this project. It was used to suggest code completions, draft helper functions, generate initial unit tests, and improve documentation. All generated code was reviewed, modified where necessary, and tested by the author. Copilot was not used as a source of scientific conclusions, and the author remains responsible for the correctness of the implementation and interpretation.

For larger projects, it may also be useful to keep a short AI-use log. This can be a simple text file in the repository, for example `AI_USE.md`, where the student records the main ways Copilot was used. The log does not need to contain every accepted inline suggestion. It should document substantial uses, such as generating an initial implementation, debugging a numerical issue, refactoring several files, or drafting tests.

A minimal log could look like this:

```
# AI use log

Tool: GitHub Copilot
Project: Driven two-level quantum system

Use 1:
Copilot was used to draft an initial function for computing
state fidelity between two complex vectors.
Verification: checked against identical, orthogonal, and phase-shifted states.

Use 2:
Copilot was used to suggest pytest tests for the Rabi-model fitting code.
Verification: tests were edited manually and run on synthetic data
```

with known parameters.

Use 3:

Copilot Chat was used to review the code for possible issues involving normalization, complex conjugation, array shapes, and units.

Verification: suggestions were inspected manually before changes were made.

This kind of documentation helps avoid a common risk of AI-assisted work: the impression that because the tool gives fluent explanations or plausible code, the user understands more than they actually do. [4] describe this as an *illusion of understanding* in scientific research. In practice, the response should be disciplined use: ask Copilot to state assumptions, generate tests, identify limitations, and explain alternatives, but do not outsource scientific judgment to it.

2.20 Summary

GitHub Copilot is most useful when it is used as a guided assistant rather than as an automatic solution generator. Inline suggestions help with local code completion. Copilot Chat helps with planning, explanation, debugging, and review. Inline chat is useful for targeted changes to selected code. Edit mode and agent mode can help with larger tasks, but they require careful inspection of proposed changes. Custom instructions and prompt files make Copilot more consistent across a project.

For students, the main lesson is that Copilot changes the workflow. It can make programming faster, but it also makes validation more important. A good user writes clear prompts, constrains the task, asks for assumptions, generates tests, reviews diffs, and checks the scientific meaning of the result. Used this way, Copilot becomes a practical tool for moving from scientific question to reproducible computational experiment.

2.21 A caveat on trust

Copilot, similar to other LLMs, should be used prudently, especially when it generates more than a few lines of code. Large code generations can look coherent while hiding subtle mistakes in logic, units, conventions, numerical methods, or scientific assumptions. This is particularly dangerous in scientific programming, where code may run without implementing the intended model. A generated simulation may use the wrong sign convention for a Hamiltonian, forget to normalize a quantum state, mix angular frequency with ordinary frequency, or compare models using an inappropriate validation procedure. The user should therefore avoid accepting large blocks of generated code without inspection. A better workflow is to generate small pieces, read each piece, test it on simple cases, and verify the scientific meaning before moving on.

This is also a broader epistemic issue. Messeri and Crockett ([4]) argue that AI tools in science can create *illusions of understanding*: researchers may feel that they understand more than they actually do because the tool produces fluent explanations, plausible code, or apparently objective analyses. They warn that widespread use of AI can make science more productive in volume while also making it more vulnerable to errors, monocultures, and reduced conceptual diversity. In this guide, the practical consequence is simple: Copilot should support understanding, not replace it. Students should ask the tool to state assumptions, generate tests, identify limitations, and explain alternatives, but they should not outsource judgment to it. The final responsibility for correctness, interpretation, and reproducibility remains with the human researcher.

References

- [1] GitHub Docs. About individual github copilot plans and benefits. <https://docs.github.com/en/copilot/concepts/billing/individual-plans>, 2026. Accessed 2026-07-29.
- [2] GitHub Docs. Github education for students. <https://docs.github.com/en/education/about-github-education/github-education-for-students>, 2026. Accessed 2026-07-29.
- [3] GitHub Docs. Quickstart for github copilot. <https://docs.github.com/en/copilot/get-started/quickstart>, 2026. Accessed 2026-07-29.
- [4] Lisa Messeri and M. J. Crockett. Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627:49–58, 2024.
- [5] Visual Studio Code Documentation. Set up github copilot in vs code. <https://code.visualstudio.com/docs/setup/copilot>, 2026. Accessed 2026-07-29.